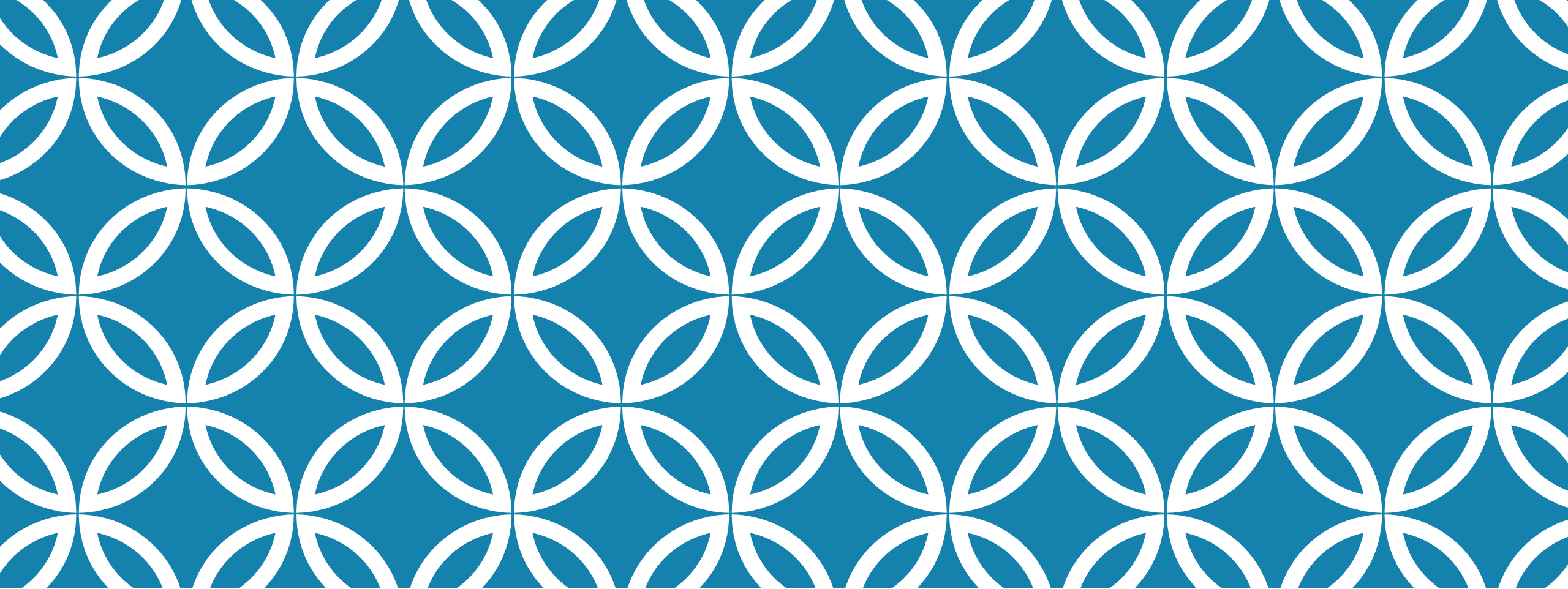




STATIC PROGRAM ANALYSIS

Lecture 11 – Inter-procedural
analysis

INTRA VS INTER-PROCEDURAL ANALYSIS

Intra-procedural

- Data computation is bounded to a single method
- Over-approximates calls to methods (returns top)
- Assumes all possible values to its entry nodes (input values are top)

Inter-procedural

- Data computation propagates from one method to another and back
- Gives a callee info about its computation (the callee's input nodes could be not tops)
- Receives info for the callee, the calls to methods could be better approximates

Used information from a call graph

EXAMPLE OF INTRA VS INTER

```
private static int x;  
private static int y;
```

```
static int m1(int z) {  
1: x = x + 1;  
2: if(x < z) {  
3: x = x * 2;  
4: y = y - 1;  
}  
5: x = y - 2;  
6: return x;  
}
```

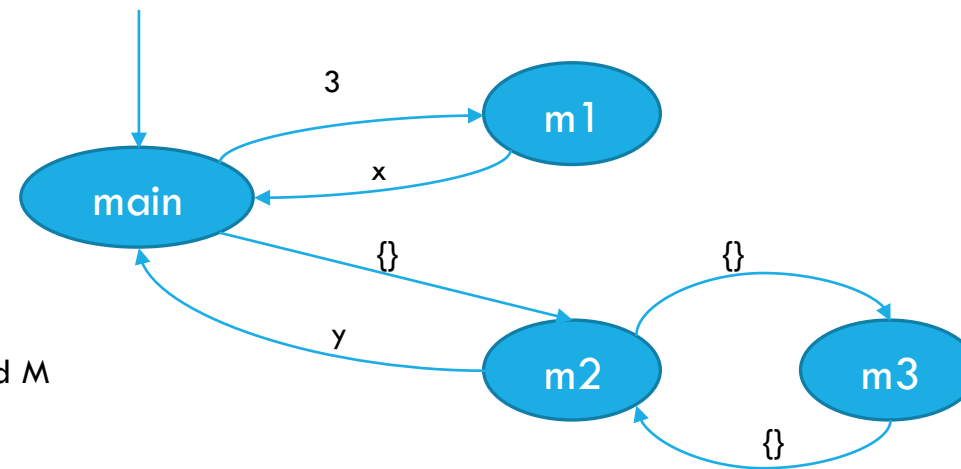
```
static int m2() {  
1: x = x - 1;  
2: m3();  
3: x = y - 2;  
return x;  
}  
  
static void m3() {  
1: while(x != y) {  
2: y = x - y;  
}  
}
```

```
private static int x;  
private static int y;  
  
public static void main(String[] args) {  
1: x = 0  
2: y = 0;  
3: y = m1(3);  
4: x = m2();  
}
```

CALL GRAPH

Algorithm for building a call graph

- Reachability analysis of M
 - for each statement S in M
 - if S is a method invocation C
 - get method name N of C
 - resolve N to an existing method M'
 - add an edge between M and M'
 - add edges between all return statements of M' and M
 - end if
 - end for each



RD data is (var, line) is
now annotated by method
(var, line, method)

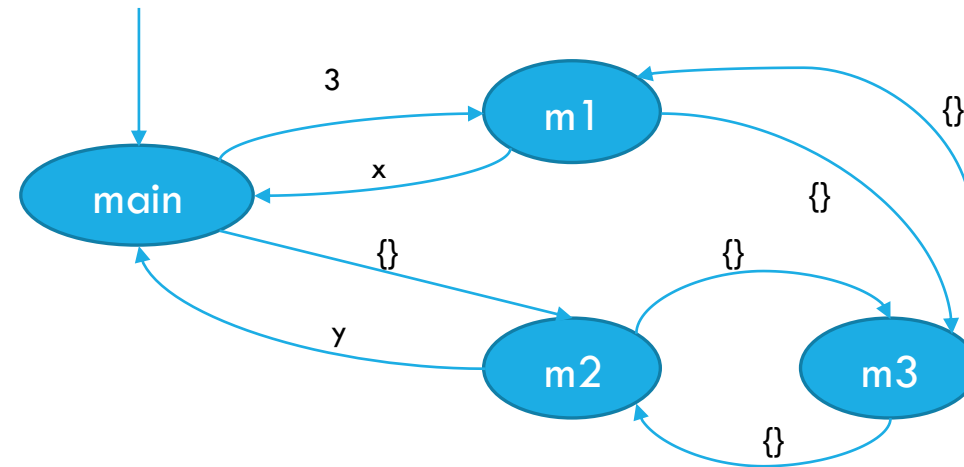
Resolving static is straightforward

Resolving dynamic: all methods with the same name N

Let's compute RD for this program

CALL GRAPH AND RD ANALYSIS

```
static int m1(int z) {  
  x = x + 1;  
  if(x < z) {  
    x = x * 2;  
    y = y - 1;  
  }  
  m3();  
  x = y - 2;  
  return x;  
}
```



How would RD look here?

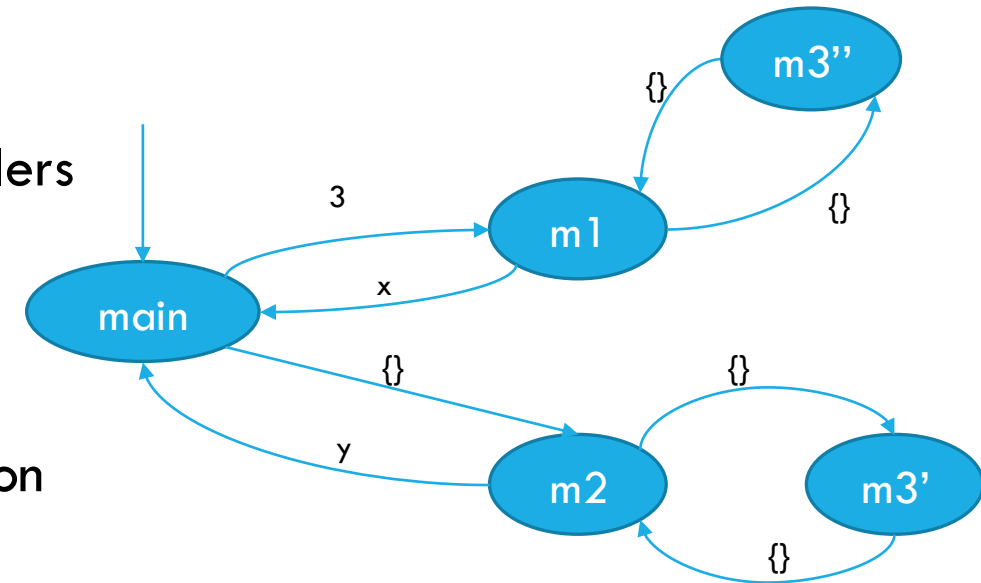
What additional operations should we define?

CONTEXT SENSITIVITY

Calling the same method from two different callers

- Create a copy for each of them
- Limit the number of copies
- k-level context sensitivity

Trade-off between k value and analysis precision



Let's look at WALA's
Context-insensitive
Context-sensitive

OBJECT SENSITIVITY

Dealing with dynamic dispatches:

- Create an edge to all method with the same signatures
- Grossly over-approximates (e.g., toString or get(int val), set(int val))

Resolve method hierarchy dependencies

- If the type of a variable is “A”
 - Consider only methods of name N from
 - subclasses?
 - superclasses?

NEXT TIME

- Work more with WALA
- Interprocedural Finite Distributive Subset problems (IFDS)

Precise Interprocedural Dataflow Analysis via Graph Reachability, Reps, Horwitz, Sagiv, POPL 1995

Setting:

- lattice of abstract states: $\text{States} = P(D)$ where D is a finite set (i.e., a powerset lattice)
- all transfer functions, $f \vee : \text{States} \rightarrow \text{States}$, are distributive

Great idea #1:

- such constraints can be represented compactly!
- distributivity closed under composition and least upper bound, so function summaries can also be represented compactly and without loss of precision!

Great idea #2:

- tabulation solver (pre-analysis + main analysis)
- can be made demand driven